## Unit III – Constructors, Destructors & Operator Overloading: Answers

Debugging (Section A) and Pseudo-Code (Section B). Corrected C++ programs were compiled with g++ and their outputs are real.

**Section A – Debugging Questions**

**Question 1: Copy Constructor Pass-by-Value Trap**

**1.**Exact compiler error (Line 17, Compilation Error)

error: invalid constructor; you probably meant 'DynamicString (const DynamicString&)'

(Wording differs slightly between compilers; Clang says copy constructor must pass its first argument by reference.)

**2. Why pass-by-value is infinite recursion**

Passing source by value means the parameter must itself be initialized by copying the argument, and copying a DynamicString is done by the copy constructor. So calling the copy constructor requires first calling the copy constructor to build its parameter, which requires another copy to build that parameter, and so on without end. Because this recursion could never terminate, the C++ standard makes such a declaration ill-formed, and the compiler rejects it at compile time instead of overflowing the stack at run time.

**3. Correct signature**

DynamicString(const DynamicString &source);

- & (reference) binds the parameter to the original object, so no copy is made and the recursion disappears.
- const promises the source is not modified, and lets the constructor accept const objects and temporaries (rvalues), as in DynamicString s2 = makeString();.

**4. Complete corrected code**

```
#include <iostream>
#include <cstring>
class DynamicString {
private:
   char *buffer;
   int length;
```

```
public:
    DynamicString(const char *str) {
        length = std::strlen(str);
        buffer = new char[length + 1];
        std::strcpy(buffer, str);
    }
    DynamicString(const DynamicString &source) {
        length = source.length;
        buffer = new char[length + 1];
        std::strcpy(buffer, source.buffer);
    }
    ~DynamicString() { delete[] buffer; }
    void print() const { std::cout << buffer << std::endl; }
};
int main() {
    DynamicString s1("Antigravity C++");
    DynamicString s2 = s1;
    s2.print();
    return 0;
}
```

Dry run: s1 allocates a 16-byte buffer (15 characters + '\0'). s2 = s1 calls the copy constructor, which allocates a separate 16-byte buffer and copies the text. Both destructors later free their own buffers.

**Output:**

Antigravity C++

**Question 2: Shallow Copy & Double Free**

**1. Address trace and default copy constructor**

The compiler-synthesized copy constructor does a **member-wise (shallow) copy**: it copies the pointer value, not the data it points to. So the copy and the original hold the same address.

| Moment | track.samples | buf.samples (parameter) |
|---|---|---|
| Before processAudio | 0xA (allocated) | – |
| During processAudio | 0xA | 0xA (same block, shared) |
| After processAudio returns (buf destroyed) | 0xA now freed (dangling) | destroyed |
| End of main (track destroyed) | delete[] 0xA again → double free | – |

**2. Why the crash**

When processAudio returns, the parameter buf is destroyed and its destructor does delete[] samples, releasing block 0xA. track still points at 0xA. Printing track.samples[0] reads freed memory (undefined behaviour). At the end of main, track's destructor calls delete[] on the same address a second time, and the allocator aborts with free(): double free detected.

**3. Deep copy constructor and corrected code**

```
#include <iostream>
class AudioBuffer {
public:
   int *samples;
   int capacity;
   AudioBuffer(int cap) : capacity(cap) {
      samples = new int[capacity];
      for (int i = 0; i < capacity; ++i) samples[i] = i * 10;
      std::cout << "Buffer allocated at: " << samples << std::endl;
   }
   AudioBuffer(const AudioBuffer &other) : capacity(other.capacity) {
      samples = new int[capacity];
      for (int i = 0; i < capacity; ++i) samples[i] = other.samples[i];
      std::cout << "Deep copy allocated at: " << samples << std::endl;
   }
```

```
    ~AudioBuffer() {
        std::cout << "Deallocating buffer at: " << samples << std::endl;
        delete[] samples;
    }
};
void processAudio(AudioBuffer buf) {
    std::cout << "Processing sample 0: " << buf.samples[0] << std::endl;
}
int main() {
    AudioBuffer track(5);
    processAudio(track);
    std::cout << "Back in main, sample 0: " << track.samples[0] << std::endl;
    return 0;
}
```

**4. Verified output**

Addresses differ per run; the key point is that the two allocations are distinct and each is freed exactly once.

**Output:**

```
Buffer allocated at: 0x55f46ca282b0
Deep copy allocated at: 0x55f46ca292e0
Processing sample 0: 0
Deallocating buffer at: 0x55f46ca292e0
Back in main, sample 0: 0
Deallocating buffer at: 0x55f46ca282b0
```

**Question 3: Stream Insertion << as a Member Function**

**1. Translation of member operators**

For a member binary operator, a @ b is rewritten as a.operator@(b): the **left operand is the object the member is called on**. For std::cout << f1 the left operand is std::cout (an ostream), so the compiler looks for ostream::operator<<(Fraction) or a non-member operator<<(ostream&,

Fraction). A member of Fraction is never considered, and we cannot add members to std::ostream. Hence: no match for 'operator<<'.

**2. Why f1 << std::cout compiles and why it is bad**

Here the left operand is a Fraction, so it becomes f1.operator<<(std::cout), which matches the member. It works but reverses the universally expected stream syntax, breaks chaining (cout << a << b), and confuses every reader, so it is unacceptable in production code.

**3-4. Friend non-member fix and output**

```
#include <iostream>
class Fraction {
private:
   int numerator;
   int denominator;
public:
   Fraction(int n, int d) : numerator(n), denominator(d) {}
   friend std::ostream& operator<<(std::ostream &os, const Fraction &f);
};
std::ostream& operator<<(std::ostream &os, const Fraction &f) {
   os << f.numerator << "/" << f.denominator;
   return os;
}
int main() {
   Fraction f1(3, 4);
   std::cout << f1 << std::endl;
   return 0;
}
```

The friend function gets access to private members, takes the stream by non-const reference, the object by const reference, and returns the stream to allow chaining (std::cout << f1 << std::endl).

**Output:**

**Question 4: Prefix vs Postfix Increment Signatures**

**1. Standard signatures**

| Operator | Signature | Returns |
|---|---|---|
| Prefix ++obj | StepCounter& operator++() | reference to the modified object (*this) |
| Postfix obj++ | StepCounter operator++(int) | copy of the old value, by value |

**2. Dummy int**

Both operators have the same name operator++ and differ only in parameters. The unused int parameter is a language-defined marker: the compiler passes 0 and picks the (int) version for obj++ and the no-parameter version for ++obj. It carries no data.

**3. Why reference vs value**

Prefix increments, then yields the same, updated object, so returning a reference is efficient and makes ++(++x) work on the same object. Postfix must return the value before incrementing; that old value is a local temporary that dies when the function ends, so it can only be returned by value (returning a reference to it would dangle). This is also why postfix is slower than prefix for class types.

**In the buggy code:** the version without int (called by ++c1) was written as postfix (returns the old copy 10), and the (int) version as prefix. So c2 = ++c1 received the old value 10.

**4. Corrected code**

```
#include <iostream>
class StepCounter {
private:
   int steps;
public:
   StepCounter(int s = 0) : steps(s) {}
   StepCounter& operator++() {          // prefix
      ++steps;
      return *this;
```

```
    }
    StepCounter operator++(int) {          // postfix (dummy int)
        StepCounter temp = *this;
        ++steps;
        return temp;
    }
    void display() const { std::cout << "Steps: " << steps << std::endl; }
};
int main() {
    StepCounter c1(10);
    StepCounter c2 = ++c1;
    c2.display();
    c1.display();
    StepCounter c3 = c1++;
    c3.display();
    c1.display();
    return 0;
}
```

| Statement | c1 | Object returned |
|---|---|---|
| c1(10) | 10 | – |
| c2 = ++c1 | 11 | c2 = 11 (new value) |
| c3 = c1++ | 12 | c3 = 11 (old value) |

**Output:**

```
Steps: 11
Steps: 11
Steps: 11
Steps: 12
```

**Question 5: Operator Overloading Rules**

**1. Operators that cannot be overloaded**

. (member access), .* (pointer-to-member access), :: (scope resolution), ?: (ternary conditional), sizeof, typeid (and the cast keywords such as static_cast, plus alignof/noexcept operators; the preprocessor tokens # and ## are also not overloadable).

operator. is prohibited because the dot is how the language itself resolves members of a class; a.b must always mean the real member b of a. Overloading it would make it impossible to reliably access an object's own members. The right-hand side of . is a member name, not an expression/value, so it cannot be passed as a function argument. :: and sizeof operate on names/types at compile time, hence also excluded.

**2. Line 14**

At least one operand of an overloaded operator must be a **user-defined type** (class, struct or enum). operator+(int,int) would change the meaning of built-in arithmetic and break all existing code, so the compiler rejects it: 'int operator+(int, int)' must have an argument of class or enumerated type.

**3. Inventing ****

No. You can overload only the existing operator tokens. Inventing symbols such as ** is illegal (** lexes as two tokens, * and *). Overloading also cannot change **precedence**, **associativity** or **arity** (number of operands) of an operator: a binary + stays binary, and with the same precedence as built-in +. Use a named method (power) for exponentiation.

**4. Corrected code**

```
#include <iostream>
#include <cmath>
class MathVector {
public:
   double x, y;
   MathVector(double a, double b) : x(a), y(b) {}
   bool operator==(const MathVector &other) const {
      return (x == other.x && y == other.y);
```

```
    }
    MathVector operator+(const MathVector &other) const {
        return MathVector(x + other.x, y + other.y);
    }
    MathVector power(int p) const {
        return MathVector(std::pow(x, p), std::pow(y, p));
    }
};
int main() {
    MathVector v1(2.0, 3.0), v2(2.0, 3.0);
    MathVector sum = v1 + v2;
    MathVector sq = v1.power(2);
    std::cout << std::boolalpha << "v1 == v2: " << (v1 == v2) << std::endl;
    std::cout << "Sum: (" << sum.x << ", " << sum.y << ")" << std::endl;
    std::cout << "Power 2: (" << sq.x << ", " << sq.y << ")" << std::endl;
    return 0;
}
```

**Output:**

v1 == v2: true

Sum: (4, 6)

Power 2: (4, 9)

**Section B – Pseudo-Code Questions**

**Question 1: DynStr – Deep Copy & Overloaded +**

**Design overview**

- Rule of three: destructor frees heap memory, copy constructor deep copies, so no two objects share a buffer.
- Operator + builds a new buffer of size (len1 + len2 + 1) and returns a new object.
- Memory is NULL-ed after release to avoid dangling pointers.

```
CLASS DynStr
  PRIVATE:
    POINTER TO CHARACTER dataPtr
    INTEGER charCount
  PUBLIC:
    CONSTRUCTOR DynStr(STRING sourceString)
      SET charCount = LENGTH(sourceString)
      SET dataPtr = ALLOCATE ARRAY OF CHARACTER [charCount + 1]
      INTEGER i
      FOR i = 0 TO charCount - 1 STEP 1 DO
        SET dataPtr[i] = sourceString[i]
      ENDFOR
      SET dataPtr[charCount] = NULL_CHAR
    ENDCONSTRUCTOR

    CONSTRUCTOR DynStr(REF other AS DynStr)          // deep copy
      SET charCount = other.charCount
      SET dataPtr = ALLOCATE ARRAY OF CHARACTER [charCount + 1]
      INTEGER i
      FOR i = 0 TO charCount STEP 1 DO
        SET dataPtr[i] = other.dataPtr[i]
      ENDFOR
    ENDCONSTRUCTOR

    DESTRUCTOR ~DynStr()
      IF dataPtr NOT= NULL THEN
        DEALLOCATE dataPtr
        SET dataPtr = NULL
      ENDIF
    ENDDESTRUCTOR
```

```
    OPERATOR +(REF rhs AS DynStr) RETURNS DynStr
      INTEGER total, i, j
      SET total = charCount + rhs.charCount
      STRING temp = ALLOCATE ARRAY OF CHARACTER [total + 1]
      FOR i = 0 TO charCount - 1 STEP 1 DO
        SET temp[i] = dataPtr[i]
      ENDFOR
      FOR j = 0 TO rhs.charCount - 1 STEP 1 DO
        SET temp[charCount + j] = rhs.dataPtr[j]
      ENDFOR
      SET temp[total] = NULL_CHAR
      DynStr result(temp)              // builds its own buffer
      DEALLOCATE temp
      RETURN result
    ENDOPERATOR

    METHOD Display()
      DISPLAY dataPtr
    ENDMETHOD
ENDCLASS

BEGIN Main
  DynStr strA("Hello ")
  DynStr strB("World!")
  DynStr strC = strA + strB
  strC.Display()
END
```

Dry-run trace (strA = "Hello ", strB = "World!")

| Step | Action | charCount | Heap buffer | Characters copied |
|---|---|---|---|---|
| 1 | strA constructed | 6 | bufA, 7 bytes | 'Hello ' + NUL |

| Step | Action | charCount | Heap buffer | Characters copied |
|---|---|---|---|---|
| 2 | strB constructed | 6 | bufB, 7 bytes | 'World!' + NUL |
| 3 | operator+ : total = 6 + 6 | 12 | temp, 13 bytes | 6 from strA, then 6 from strB, NUL at index 12 |
| 4 | result constructed from temp | 12 | bufC, 13 bytes | 'Hello World!' + NUL (12 + 1) |
| 5 | temp deallocated | – | temp freed | – |
| 6 | strC receives result | 12 | bufC | (copy elided or deep-copied; any extra buffer is freed) |
| 7 | Display | – | – | prints Hello World! |
| 8 | End of Main: destructors strC, strB, strA | – | bufC, bufB, bufA all freed | no leak |

**Question 2: Complex Number Class**

**CLASS Complex**

```
  PRIVATE:
    REAL real, imag
  PUBLIC:
    CONSTRUCTOR Complex()
      SET real = 0.0
      SET imag = 0.0
    ENDCONSTRUCTOR
    CONSTRUCTOR Complex(REAL r)
      SET real = r
      SET imag = 0.0
    ENDCONSTRUCTOR
    CONSTRUCTOR Complex(REAL r, REAL i)
      SET real = r
      SET imag = i
    ENDCONSTRUCTOR
```

```
    OPERATOR +(REF other AS Complex) RETURNS Complex
      RETURN Complex(real + other.real, imag + other.imag)
    ENDOPERATOR

    OPERATOR *(REF other AS Complex) RETURNS Complex
      REAL r, i
      SET r = (real * other.real) - (imag * other.imag)
      SET i = (real * other.imag) + (imag * other.real)
      RETURN Complex(r, i)
    ENDOPERATOR

    OPERATOR ==(REF other AS Complex) RETURNS BOOLEAN
      RETURN (real = other.real) AND (imag = other.imag)
    ENDOPERATOR

    METHOD Display()
      IF imag >= 0 THEN
        DISPLAY real, " + ", imag, "i"
      ELSE
        DISPLAY real, " - ", ABS(imag), "i"
      ENDIF
    ENDMETHOD
ENDCLASS

BEGIN Main
  Complex c1(3, 2)
  Complex c2(1, 4)
  Complex p = c1 * c2
  p.Display()
END
```

Trace: C1 = 3 + 2i, C2 = 1 + 4i (a=3, b=2, c=1, d=4)

| Step | Formula | Computation | Value |
|---|---|---|---|
| 1 | ac | 3 × 1 | 3 |
| 2 | bd | 2 × 4 | 8 |
| 3 | real = ac − bd | 3 − 8 | −5 |
| 4 | ad | 3 × 4 | 12 |
| 5 | bc | 2 × 1 | 2 |
| 6 | imag = ad + bc | 12 + 2 | 14 |
| 7 | Result | | −5 + 14i |

Display prints: **-5 + 14i**.

**Question 3: Currency Class with Stream I/O & Unary Minus**

**Design overview**

- Value is normalized after every construction/read: total = dollars×100 + cents, then dollars = total / 100 and cents = total MOD 100 (integer division truncating toward zero, so both parts carry the same sign for negative values).
- Unary minus flips the sign of both parts. Output pads cents to two digits and puts the minus sign before the $.

```
CLASS Currency
  PRIVATE:
    INTEGER dollars, cents
    METHOD Normalize()
      INTEGER total
      SET total = dollars * 100 + cents
      SET dollars = total / 100          // integer division
      SET cents   = total MOD 100        // same sign as total
    ENDMETHOD
  PUBLIC:
    CONSTRUCTOR Currency(INTEGER d = 0, INTEGER c = 0)
      SET dollars = d
      SET cents = c
```

```
    Normalize()
  ENDCONSTRUCTOR

  OPERATOR -() RETURNS Currency
    Currency result
    SET result.dollars = -dollars
    SET result.cents   = -cents
    RETURN result
  ENDOPERATOR

  FRIEND OPERATOR <<(REF outStream, REF cur AS Currency) RETURNS REF outStream
    IF cur.dollars < 0 OR cur.cents < 0 THEN
      WRITE outStream, "-$", ABS(cur.dollars), "."
    ELSE
      WRITE outStream, "$", cur.dollars, "."
    ENDIF
    IF ABS(cur.cents) < 10 THEN
      WRITE outStream, "0"
    ENDIF
    WRITE outStream, ABS(cur.cents)
    RETURN outStream
  ENDOPERATOR

  FRIEND OPERATOR >>(REF inStream, REF cur AS Currency) RETURNS REF inStream
    READ inStream, cur.dollars, cur.cents
    cur.Normalize()
    RETURN inStream
  ENDOPERATOR
ENDCLASS

BEGIN Main
```

```
  Currency cur
  READ cin >> cur
  DISPLAY cout << cur
  Currency neg = -cur
  DISPLAY cout << neg
END
```

Trace: input d = 5, c = 275

| Step | Operation | Values |
|---|---|---|
| 1 | Read | dollars = 5, cents = 275 |
| 2 | total = 5×100 + 275 | 775 |
| 3 | dollars = 775 / 100 | 7 |
| 4 | cents = 775 MOD 100 | 75 |
| 5 | Print cur | $7.75 |
| 6 | -cur → dollars = −7, cents = −75 | (−7, −75) |
| 7 | Print neg (negative → "-$", ABS values, cents ≥ 10 so no padding) | -$7.75 |

**Question 4: Polynomial with Subscript Operator**

**CLASS Polynomial**

```
  PRIVATE:
    INTEGER degree
    POINTER TO REAL coeffArray
    REAL dummy                          // returned on a bad index
  PUBLIC:
    CONSTRUCTOR Polynomial(INTEGER deg)
      SET degree = deg
      SET coeffArray = ALLOCATE ARRAY OF REAL [degree + 1]
      INTEGER i
      FOR i = 0 TO degree STEP 1 DO
        SET coeffArray[i] = 0.0
```

```
  ENDFOR
  SET dummy = 0.0
ENDCONSTRUCTOR

CONSTRUCTOR Polynomial(REF other AS Polynomial)       // deep copy
  SET degree = other.degree
  SET coeffArray = ALLOCATE ARRAY OF REAL [degree + 1]
  INTEGER i
  FOR i = 0 TO degree STEP 1 DO
    SET coeffArray[i] = other.coeffArray[i]
  ENDFOR
  SET dummy = 0.0
ENDCONSTRUCTOR

DESTRUCTOR ~Polynomial()
  DEALLOCATE coeffArray
ENDDESTRUCTOR

OPERATOR [](INTEGER index) RETURNS REF REAL
  IF index >= 0 AND index <= degree THEN
    RETURN REF coeffArray[index]
  ELSE
    DISPLAY "Error: power out of range"
    SET dummy = 0.0
    RETURN REF dummy
  ENDIF
ENDOPERATOR

OPERATOR +(REF rhs AS Polynomial) RETURNS Polynomial
  INTEGER maxDeg, i
  IF degree >= rhs.degree THEN
```

```
      SET maxDeg = degree
    ELSE
      SET maxDeg = rhs.degree
    ENDIF
    Polynomial result(maxDeg)
    FOR i = 0 TO maxDeg STEP 1 DO
      REAL a = 0.0, b = 0.0
      IF i <= degree THEN SET a = coeffArray[i] ENDIF
      IF i <= rhs.degree THEN SET b = rhs.coeffArray[i] ENDIF
      SET result[i] = a + b
    ENDFOR
    RETURN result
  ENDOPERATOR
ENDCLASS

BEGIN Main
  Polynomial p1(2), p2(3)
  p1[0] <- 5   p1[1] <- 2   p1[2] <- 3
  p2[0] <- 7   p2[1] <- 0   p2[2] <- 1   p2[3] <- 4
  Polynomial p3 = p1 + p2
END
```

Trace: P1 = $3x^2 + 2x + 5$ (coeffs [5, 2, 3]), P2 = $4x^3 + x^2 + 7$ (coeffs [7, 0, 1, 4])

| i (power) | P1 coeff | P2 coeff | result[i] |
|---|---|---|---|
| 0 | 5 | 7 | 12 |
| 1 | 2 | 0 | 2 |
| 2 | 3 | 1 | 4 |
| 3 | – (i > 2, so 0) | 4 | 4 |

maxDeg = max(2, 3) = 3. Result: **$4x^3 + 4x^2 + 2x + 12$**.

**Question 5: Vector2D – Prefix/Postfix ++ and +=**

**CLASS Vector2D**

```
 PRIVATE:
  REAL x, y
 PUBLIC:
  CONSTRUCTOR Vector2D(REAL px = 0.0, REAL py = 0.0)
   SET x = px
   SET y = py
  ENDCONSTRUCTOR

  OPERATOR ++() RETURNS REF Vector2D          // prefix
   SET x = x + 1.0
   SET y = y + 1.0
   RETURN REF THIS
  ENDOPERATOR

  OPERATOR ++(INTEGER dummy) RETURNS Vector2D // postfix
   Vector2D snapshot = THIS                   // copy of current state
   SET x = x + 1.0
   SET y = y + 1.0
   RETURN snapshot                            // by value
  ENDOPERATOR

  OPERATOR +=(REF rhs AS Vector2D) RETURNS REF Vector2D
   SET x = x + rhs.x
   SET y = y + rhs.y
   RETURN REF THIS
  ENDOPERATOR

  METHOD Display()
   DISPLAY "(", x, ", ", y, ")"
```

```
    ENDMETHOD
ENDCLASS

BEGIN Main
  Vector2D v1(5.0, 8.0), v2, v3
  SET v2 = ++v1
  SET v3 = v1++
  v1.Display()   v2.Display()   v3.Display()
END
```

Trace (v1 starts at (5.0, 8.0))

| Statement | v1 | v2 | v3 | Explanation |
|---|---|---|---|---|
| start | (5, 8) | (0, 0) | (0, 0) | defaults for v2, v3 |
| v2 <- ++v1 | (6, 9) | (6, 9) | (0, 0) | prefix: increment first, return updated object |
| v3 <- v1++ | (7, 10) | (6, 9) | (6, 9) | postfix: return snapshot (6,9), then increment v1 |

Final: v1 = (7.0, 10.0), v2 = (6.0, 9.0), v3 = (6.0, 9.0).